자동 추출된 프로토콜 상태 머신의 의미론적 충실도 평가

Hassan Asghar[1], 이현우[1]
[1]한국에너지공과대학교 에너지공학부

hassanasghar@kentech.ac.kr, hwlee@kentech.ac.kr

Evaluating the Semantic Fidelity of Automatically Extracted Protocol State Machines

Hassan Asghar[1], Hyunwoo Lee[1]
[1]Department of Energy Engineering, Korea Institute of Energy Technology
hassanasghar@kentech.ac.kr, hwlee@kentech.ac.kr

Abstract

Automatically extracted protocol state machines are often evaluated by structural correspondence, but high correspondence does not establish semantic fidelity. We review five state-machine extraction studies and perform a controlled experiment on three protocol-property fragments: TLS 1.3 Finished verification, OAuth PKCE binding, and authorization-code single use. We construct 15 finite models and compare edge F1 and condition/action matching with safety, valid-trace reachability, and observable equivalence. A security-violating variant retains perfect structural scores, while non-completing variants also retain perfect structural scores and satisfy the safety property. State renaming yields zero edge F1 despite behavioral equivalence. These results show that structural correspondence, security-property satisfaction, and behavioral preservation provide distinct forms of evidence.

Keywords: protocol state machines; model extraction; semantic fidelity; conformance; model checking

1. Introduction

Automated methods increasingly extract finite state machines (FSMs) from protocol specifications and implementations for formal verification, test generation, and fuzzing [1]. An extracted model can match source transitions while changing a guard, variable update, or allowed execution. Conversely, systematic state renaming can change labels without changing behavior. The central evaluation problem is therefore whether structural correspondence provides evidence of semantic fidelity.

We use semantic fidelity to mean preservation of source behavior relevant to the evaluation target under explicit abstraction assumptions. For a normative specification, this concerns specified or permitted behavior; for an implementation, it concerns implementation behavior. Fidelity is distinct from security-property satisfaction: a model may satisfy a security property because behavior that would violate it has been omitted.

This paper makes three contributions: a focused review of evaluation criteria in five protocol state-machine extraction studies; a controlled experiment on 15 finite models that isolates structural and behavioral changes; and a comparison of structural correspondence, security-property satisfaction, valid-trace reachability, and observable equivalence. We ask whether structural matching distinguishes security-relevant semantic changes from behavior-preserving renaming, and whether safety-property checking detects the loss of legitimate behavior.

2. Related Work

We select five representative protocol state-machine extraction studies spanning prose- and code-based inputs. Table 1 summarizes the evaluation measures they report. The selection is focused rather than exhaustive, and reported numerical scores are not treated as a common ranking.

Table 1. Evaluation measures reported in selected protocol state-machine extraction studies.

Study / source	Evaluation measures
RFCNLP [1] / RFC prose	Span and transition matching; attacker synthesis against canonical FSMs.
Hermes [2] / 3GPP prose	Condition/action matching; LTL conformance tests; security analysis.
ProtocolGPT [3] / source code	Transition precision/recall; implementation comparison; downstream fuzzing.
FlowFSM [4] / RFC prose	Transition precision, recall, and F1; manual cross-checking against RFCs.
SpecGPT [5] / 3GPP prose	State and transition F1 against manually annotated references.

Prior work combines structural and behavioral evaluation. RFCNLP discusses extracted protocol models used for attacker synthesis [1]. Hermes supplements transition-accuracy measures with 43 security-related conformance tests expressed as LTL properties [2]. ProtocolGPT evaluates state-machine inference from protocol source code and its downstream use for fuzzing [3]. FlowFSM evaluates extracted transitions against RFC-grounded references [4], while SpecGPT reports F1-based comparison against

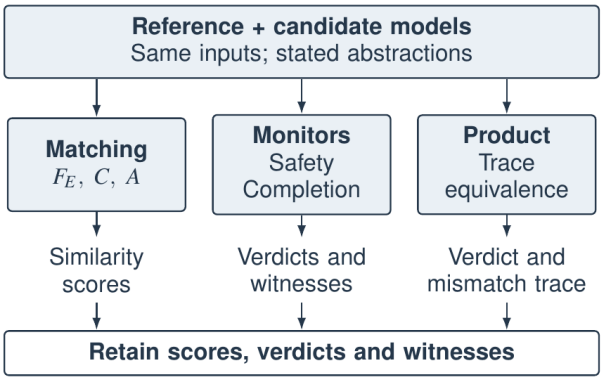
manually annotated references [5]. These checks address different notions of model correctness.

The reviewed studies also use different reference semantics: RFC-derived models, implementation-derived models, and manually annotated state machines. Their reported scores are consequently not directly comparable as a measure of semantic fidelity. We use a simplified implementation of the condition/action matching definition described in Hermes [2], adapted to our rule representation; no published extractor is rerun.

3. Experimental Method

3.1 Structural and Behavioral Evaluation

Figure 1 summarizes the complementary checks. Each model is a deterministic partial Mealy machine with finite variables. A rule contains a source state, input event, conjunctive guard, target state, output, and assignments. Unmentioned variables retain their values. An input with no enabled rule is blocked, distinct from explicit rejection.



(Fig. 1) Complementary structural and behavioral checks of the same model pair.

The coarse structural correspondence measure is edge F1:

$$F_E = \frac{2|E_R \cap E_M|}{|E_R| + |E_M|}$$

Here, E contains distinct (source, event type, target) triples. Guards, outputs, and assignments are ignored, and state names are not aligned. The measure is defined for this study.

Following Hermes [2], let G_r and A_r be the condition and action atom sets of a reference rule r. Source state and input event are treated as conditions; target state, output, and explicit assignments are treated as actions. For a candidate rule m, let $s(r,m)=|G_r \cap G_m|/|G_r|$ and $a(r,m)=|A_r \cap A_m|/|A_r|$. We compute:

$$C = \frac{1}{|R|} \sum_{r \in R} \max_{m \in M} s(r,m)$$

$$A = \frac{1}{|R|} \sum_{r \in R} \max_{m \in M} a(r,m)$$

An empty maximum is zero. Atoms match literally; guards are conjunctive. These measures quantify recovery of reference atoms but do not directly penalize unmatched candidate atoms. We apply the definition to our rule representation rather than to the complete Hermes pipeline.

Safety searches reachable states for a security-monitor violation. Valid-trace reachability searches for a legitimate successful execution; it is a non-vacuity check, not a liveness guarantee. Observable equivalence explores the synchronous product over the common finite alphabet until an output or enabledness mismatch appears, or all reachable pairs are exhausted. For these deterministic machines, this establishes equality of observable finite traces.

3.2 Security Requirements and Model Variants

The three references isolate security requirements from two protocol families and are modeled as small finite abstractions rather than complete protocol models.

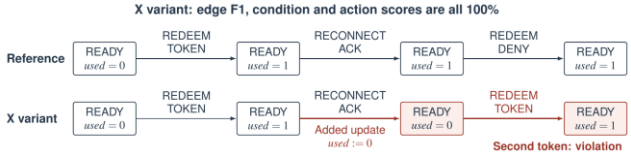
1. TLS Finished. An invalid Finished causes fatal termination, and a valid Finished is required before the handshake completes [6]. Early data is excluded, and cryptographic verification is represented as a Boolean input.

2. PKCE binding. The token endpoint issues a token only after the transformed verifier matches the code-bound challenge [7]. Two preissued codes have distinct challenges; other OAuth checks are assumed satisfied, with one redemption per run.

3. Authorization-code single use. An authorization code must not be used more than once [8]. The finite model retains the used flag across reconnects; expiry, reissuance, and token revocation are excluded from the fragment.

Each reference receives four predefined model transformations. U weakens the success condition and removes the alternative rejection rule. X corrupts a binding or lifecycle condition: TLS remains open after a fatal error, PKCE compares against the wrong code's challenge, or reconnect clears the code's used flag. B adds an immutable false conjunct to the success rule. R applies a consistent bijective renaming of control states. The set therefore contains three references, nine semantic-error variants (U/X/B), and three behavior-preserving controls (R).

Security monitors inspect input/output histories independently of internal authorization flags. The code-use monitor counts successful token responses directly. Valid completion requires a valid Finished without an earlier fatal error, a correctly bound PKCE token, or the first successful use of an authorization code. Undefined inputs remain blocked.



(Fig. 2) Trace comparison for the reference and single-use X variant; the added reconnect update permits a second token.

4. Results

In our implementation, Python 3.12.14 breadth-first exploration visits 43 monitored states across the 15 models and records shortest witnesses. SPIN 6.5.1, using generated Promela, produces the same 15 safety and 15 valid-trace

reachability verdicts. Queries with a violation stop after a witness; Python explores the full reachable graph.

Perfect structural correspondence can coexist with a security violation. Figure 2 shows the single-use X variant with F_E = C = A = 100%. Its reconnect update changes future code eligibility without changing the edge set or removing reference condition/action atoms. The monitor detects a second token issuance. Thus, matching can miss a semantic change represented by an otherwise implicit variable update.

Safety alone misses loss of valid behavior. Every B variant obtains perfect structural scores and satisfies the safety property, but none has a valid completion trace. The candidate success guard is unsatisfiable, yet the reference atoms are unchanged. Among the nine faulty variants, safety identifies six U/X variants, while valid-trace reachability identifies the three B variants. This count describes the controlled set; it is not an estimated detection rate.

State renaming preserves behavior. All R variants have zero edge F1 and reduced condition/action scores while remaining observably equivalent. Name-sensitive correspondence is therefore appropriate when label recovery is the target, but it is not a behavioral correctness criterion.

Table 2. Results for all 15 models. S, L, and Q denote safety satisfaction, valid-trace reachability, and observable equivalence.

Model group	Edge F1	Cond.	Act.	S	L	Q
Reference	100	100	100	3/3	3/3	3/3
U: weakened guard	100	83.3–88.9	83.3–88.9	0/3	3/3	0/3
X: binding/lifecycle	66.7–100	66.7–100	66.7–100	0/3	3/3	0/3
B: non-completing	100	100	100	3/3	0/3	0/3
R: state renaming	0	61.1–66.7	30.6–35.7	3/3	3/3	3/3

5. Discussion and Limitations

The experiment separates three forms of evidence: structural correspondence, security-property satisfaction, and preservation of valid behavior. A counterexample should be linked to the source requirement or implementation behavior used to interpret it. Properties or behaviors outside the modeled abstraction should be reported as unsupported rather than inferred.

For larger models, valid-trace reachability should cover relevant procedures and parameter classes. One successful PKCE exchange does not establish that every legitimate code remains usable. Likewise, selected safety properties do not establish trace equivalence. The synchronous-product check is practical here because the fragments are tiny; richer nondeterministic protocols require an explicit conformance or equivalence relation and stated abstraction.

Structural measures can be supplemented by stronger formal relations, such as trace inclusion or bisimulation, when the abstraction defines a suitable observation relation. Such relations require additional modeling assumptions and should not be treated as automatic evidence of correctness.

This controlled study has four limitations. First, the three finite abstractions are requirement fragments, not complete TLS or OAuth models. Second, the model variants are constructed examples rather than sampled extractor errors, so the experiment provides no extractor ranking or prevalence estimate. Third, the reference models and monitors have not undergone independent human validation; agreement between Python and SPIN therefore checks the implementation consistency of the encoding, not the correctness of the source interpretation. Finally, the literature selection is focused rather than exhaustive, and the matching measures depend on the stated condition/action assignment.

6. Conclusion

Structural correspondence alone is insufficient to establish the semantic fidelity of an extracted protocol state machine. In the controlled study, security-violating and non-completing variants can retain perfect structural scores, while state renaming can destroy those scores without changing observable behavior. Evaluation should therefore report structural correspondence, security-property satisfaction, and behavioral preservation as separate forms of evidence.

Acknowledgement

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Artificial Intelligence Innovation Human Resources Development (IITP-2026-RS-2026-25548323) grant funded by the Korea government (MSIT).

7. References

[1] M. L. Pacheco, M. von Hippel, B. Weintraub, D. Goldwasser, and C. Nita-Rotaru, “Automated Attack Synthesis by Extracting Finite State Machines from Protocol Specification Documents,” in Proc. 43rd IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2022, pp. 51–68, doi:10.1109/SP46214.2022.9833673.

[2] A. Al Ishtiaq, S. S. S. Das, S. M. M. Rashid, A. Ranjbar, K. Tu, T. Wu, Z. Song, W. Wang, M. Akon, R. Zhang, and S. R. Hussain, “Hermes: Unlocking Security Analysis of Cellular Network Protocols by Synthesizing Finite State Machines from Natural Language Specifications,” in Proc. 33rd USENIX Security Symposium, Philadelphia, PA, USA, 2024, pp. 4445–4462.

[3] H. Wei, L. Chen, Z. Du, Y. Wu, H. Huang, Y. Liu, G. Cheng, F. Xu, L. Wang, and B. Mao, “Unleashing the Power of LLM to Infer State Machine From the Protocol Implementation,” in Proc. 2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS), Gold Coast, Australia, 2025, pp. 1–10, doi:10.1109/IWQoS65803.2025.11143461.

[4] F. Wael, Y. Maklad, A. Hamdi, and W. Elsersy, “An Agentic Flow for Finite State Machine Extraction using Prompt Chaining,” in Proc. 2025 Intelligent Methods, Systems, and Applications (IMSA), Giza, Egypt, 2025, doi:10.1109/IMSA65733.2025.11166906.

[5] M. Zhang, R. Feng, H. Tang, Y. Zhao, J. Yang, H. Qiu, and Q. Liu, “Automated Extraction of Protocol State Machines from 3GPP Specifications with Domain-Informed Prompts and LLM Ensembles,” arXiv:2510.14348, 2025, doi:10.48550/arXiv.2510.14348.

[6] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Aug. 2018, §4.4.4.

[7] N. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, Sep. 2015, §§4.5–4.6.

[8] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Oct. 2012, §4.1.2.

[9] G. J. Holzmann, “SPIN Model Checker,” version 6.5.1, Dec. 2019.